

PYTHON AUTOMATION – LAB ASSIGNMENTS

MODULE 3: PYTHON BDD RESTFUL AUTOMATIONS

Assignments 10 – 12

Name: Swarnendu Boral

Roll No : 10

Enrollment Number: 12023052020011

Section: CSE(AI)

Overview

This document covers the three Module 3 lab assignments (Assignments 10–12), which take the Selenium Page Object Model built in Module 2 (Assignment 7) and move it onto the Behave BDD framework. Assignment 10 sets up a working Behave + Selenium project and expresses the login flow as a Gherkin scenario. Assignment 11 turns that single scenario into a data-driven Scenario Outline covering multiple username/password combinations. Assignment 12 keeps the step-definition files free of raw locators and adds an automatic failure-screenshot hook plus an Allure HTML report, mirroring what Assignment 9 did for PyTest.

Target application: <https://www.saucedemo.com/> (same site used throughout Modules 1 and 2).

Assignment 10: Selenium + Behave BDD Framework Setup

Module	Module 3 – Python BDD RESTful Automations
Assignment Title	Selenium + Behave BDD Framework Setup
Sub-topic Ref.	19.1 – Scenario 1: Implementation of Selenium Python and Behave BDD

1. Problem Statement

All login automation written so far (Modules 1–2) lives in plain Python test classes/functions. Non-technical stakeholders can't read a unittest or PyTest file to understand what is being verified. The login flow needs to be re-expressed in Behave (Python's Gherkin-based BDD framework) so the test intent reads as plain English, while the underlying automation still reuses the Page Object Model classes already built.

2. Objective

- Install and configure Behave alongside Selenium in the existing project.
- Write a login.feature file describing a successful login as Given/When/Then steps in Gherkin.
- Implement step definitions that call only the Module 2 LoginPage / InventoryPage page objects — no raw Selenium calls in the step file.
- Use Behave's environment.py hooks (before_scenario / after_scenario) for driver lifecycle, replacing unittest's setUp/tearDown and PyTest's driver fixture.
- Run the feature file from the terminal and confirm the scenario passes.

3. Tools, Software & Concepts Used

- behave (pip install behave)
- selenium
- Reused pages/base_page.py, pages/login_page.py, pages/inventory_page.py from Assignment 7
- Gherkin syntax: Feature, Scenario, Given/When/Then
- Behave hooks: before_scenario, after_scenario (features/environment.py)
- Practice site: <https://www.saucedemo.com/>

4. Implementation Steps

1. Install the required package: pip install behave selenium
2. Copy the pages/ package (base_page.py, login_page.py, inventory_page.py) from the Assignment 7 project into the root of this project, alongside a new features/ folder.
3. Create features/login.feature describing the successful-login scenario in Given/When/Then form.
4. Create features/environment.py with before_scenario and after_scenario hooks that create/quit the driver and attach the page objects to context.
5. Create features/steps/login_steps.py with @given/@when/@then step definitions that call only context.login_page and context.inventory_page methods.
6. Run the feature: behave features/login.feature --no-capture

5. Source Code

features/login.feature

```
Feature: Login to SauceDemo

  As a registered SauceDemo user
  I want to log in with valid credentials
  So that I can reach the product inventory page

  Scenario: Successful login with valid credentials
    Given the user is on the SauceDemo login page
    When the user logs in with username "standard_user" and password "secret_sauce"
    Then the user should land on the inventory page
```

features/environment.py

```
import time
from selenium import webdriver
from pages.login_page import LoginPage
from pages.inventory_page import InventoryPage

def before_scenario(context, scenario):
    """Runs before every scenario - equivalent to unittest's setUp()
    and PyTest's driver fixture setup."""
    context.driver = webdriver.Chrome()
    context.driver.maximize_window()
    context.login_page = LoginPage(context.driver)
    context.inventory_page = InventoryPage(context.driver)

def after_scenario(context, scenario):
    """Runs after every scenario - equivalent to tearDown() / fixture teardown."""
    # Optional pause so the final page stays on screen long enough for a
    # manual screenshot of the live browser, if you want one in addition
    # to the automatic driver.save_screenshot() calls in the steps below.
    time.sleep(3) # MANUAL SCREENSHOT WINDOW (optional) - browser stays open
    context.driver.quit()
```

features/steps/login_steps.py

```
from behave import given, when, then

@given('the user is on the SauceDemo login page')

def step_open_login_page(context):

    context.login_page.open()

@when('the user logs in with username "{username}" and password "{password}"')

def step_perform_login(context, username, password):

    context.login_page.login(username, password)

@then('the user should land on the inventory page')

def step_verify_inventory_loaded(context):

    loaded = context.inventory_page.is_loaded()

    # Automatic evidence capture - same pattern as Assignment 7's

    # driver.save_screenshot() call, just triggered from a Behave step now.

    context.driver.save_screenshot('picB10_inventory_loaded.png')

    assert loaded, 'Inventory page did not load after a valid login'
```

6. Output

Running `behave features/login.feature` executes the three Gherkin steps in order, logs into SauceDemo through the reused `LoginPage` object, and the Then step automatically saves `picB10_inventory_loaded.png` right before the assertion — exactly like Assignment 7, just driven from Gherkin instead of unittest.

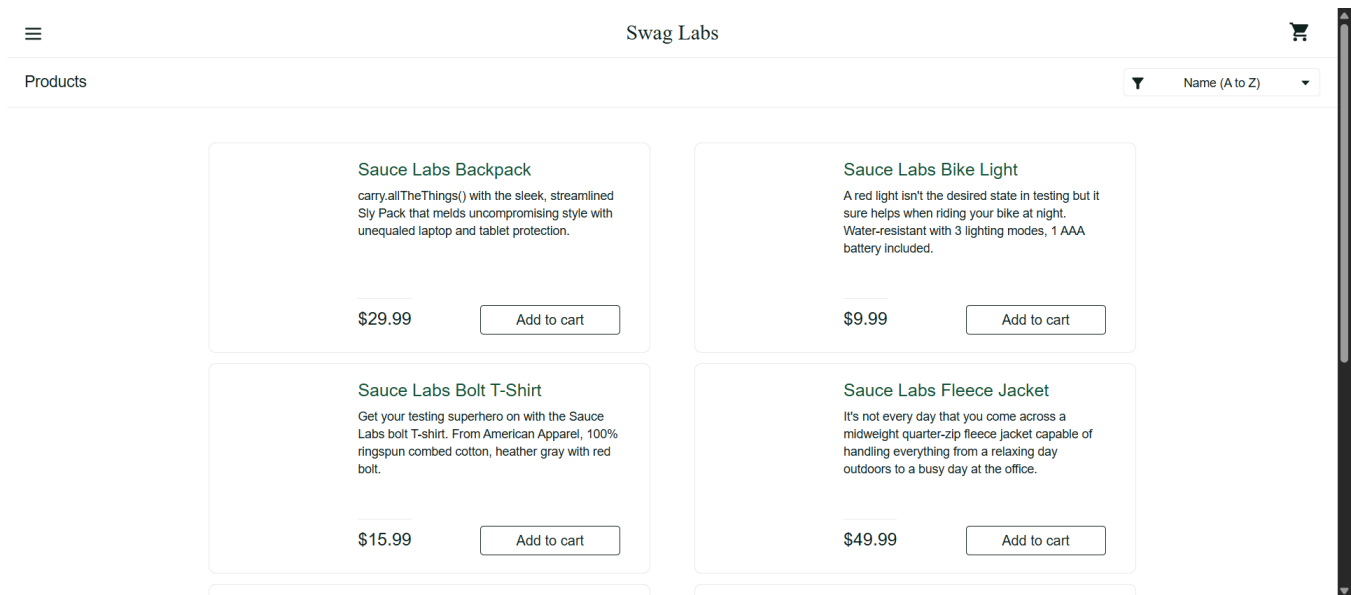


Fig B10.1 — *picB10_inventory_loaded.png (auto-captured)*

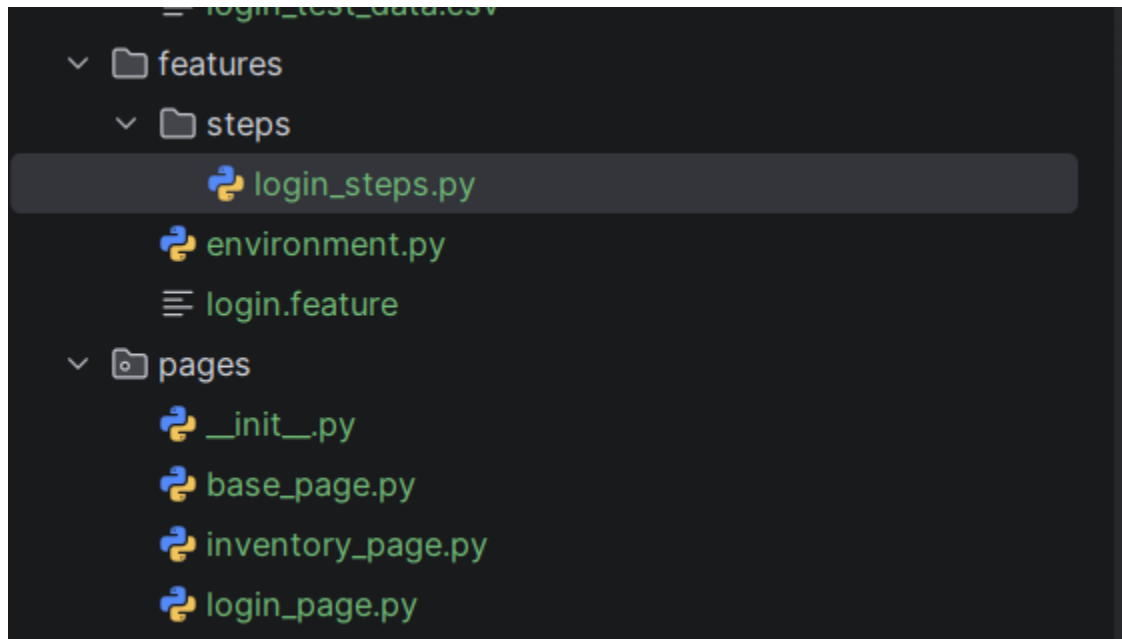


Fig B10.2 — *Project structure*

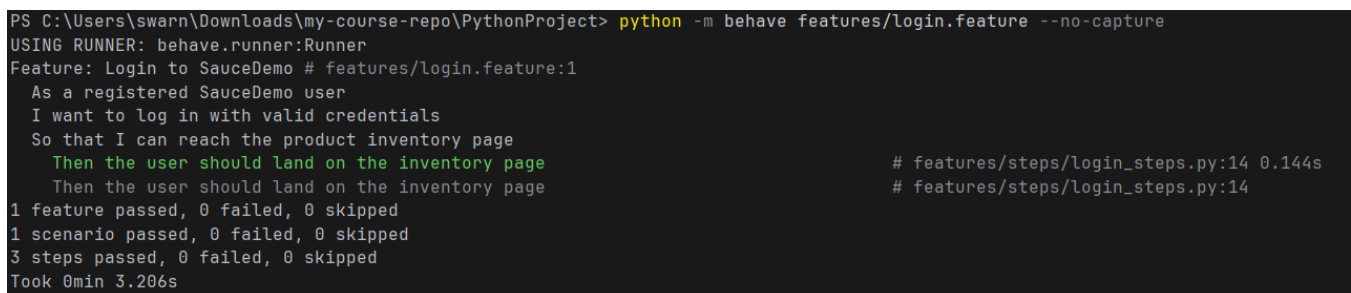


Fig B10.3 — *Terminal output*

7. Result

A working Behave + Selenium project was set up, and the Module 2 login flow now runs as a passing Gherkin scenario, with the driver lifecycle handled by environment.py hooks instead of unittest/PyTest fixtures.

8. Observation

- Because the step definitions only call context.login_page and context.inventory_page methods, the Gherkin file and the step file together read like a specification — no Selenium mechanics are visible at either layer.
- before_scenario/after_scenario run automatically for every scenario in every feature file in the project, so the driver never needs to be created or closed by hand inside a step.
- Reusing the exact same pages/ package from Assignment 7 with zero changes confirms that a well-built Page Object Model is framework-agnostic — it works under unittest, PyTest, and Behave alike.

9. Conclusion

The login flow was successfully migrated onto the Behave BDD framework: the intent of the test is now readable as plain English in login.feature, while all Selenium mechanics remain isolated inside the reused Page Object Model classes.

Assignment 11: Data-Driven Automation using Behave Scenario Outline

Module	Module 3 – Python BDD RESTful Automations
Assignment Title	Data-Driven Automation using Behave Scenario Outline
Sub-topic Ref.	19.3 – Scenario 2: Test Data-Driven Automation in Python Behave Framework

1. Problem Statement

Assignment 10's login.feature only exercises one valid username/password pair. Just like Assignment 8 needed to validate several login combinations from a CSV file under unittest, the Behave suite needs to validate the same valid / wrong-password / locked-out / unknown-user combinations — but expressed natively in Gherkin, without hand-writing a separate Scenario for every row.

2. Objective

- Replace the single hardcoded login step with a Scenario Outline using <username>, <password>, and <expected> placeholders.
- Provide an Examples table with the same four rows used in Assignment 8's login_test_data.csv (one valid case, one wrong-password case, one locked-out case, one unknown-user case).
- Implement a single Then step that branches its assertion on the expected outcome for each row.
- Automatically capture a screenshot for every row, before the assertion runs, the same way Assignment 8 did with subTest().

3. Tools, Software & Concepts Used

- behave — Scenario Outline / Examples Gherkin keywords
- Reused LoginPage / InventoryPage page objects from Assignment 7
- Reused environment.py driver hooks from Assignment 10
- Practice site: <https://www.saucedemo.com/>

4. Implementation Steps

7. Edit features/login.feature and add a new Scenario Outline block below the existing Scenario, with an Examples table of the four credential combinations.
8. Add a new Then step definition parameterized on {expected} to features/steps/login_steps.py.
9. Inside that step, save a screenshot named after the row's expected outcome, then branch the assertion: success → inventory loaded, locked_out → 'locked out' error text, invalid_credentials → 'do not match' error text.
10. Run: behave features/login.feature — Behave expands the Scenario Outline into one scenario per Examples row automatically.

5. Source Code

features/login.feature (Scenario Outline added)

```
Scenario: Successful login with valid credentials
  Given the user is on the SauceDemo login page
```

```
When the user logs in with username "standard_user" and password "secret_sauce"
Then the user should land on the inventory page
```

Scenario Outline: Login attempts with different credential combinations

```
Given the user is on the SauceDemo login page
```

```
When the user logs in with username "<username>" and password "<password>"
```

```
Then the login outcome should be "<expected>"
```

Examples: Credential combinations

username	password	expected
standard_user	secret_sauce	success
standard_user	wrong_password	invalid_credentials
locked_out_user	secret_sauce	locked_out
not_a_real_user	secret_sauce	invalid_credentials

features/steps/login_steps.py (new step appended)

```
@then('the login outcome should be "{expected}"')
def step_verify_login_outcome(context, expected):

    # Automatic per-row evidence capture, taken BEFORE the assertion so a
    # screenshot exists for every row regardless of pass/fail - same idea
    # as the per-row screenshot in Assignment 8's DDT test.

    context.driver.save_screenshot(f'picB11_{expected}_result.png')

    if expected == 'success':

        assert context.inventory_page.is_loaded(), \

            'Expected successful login but inventory page did not load'

    elif expected == 'locked_out':

        assert 'locked out' in context.login_page.get_error_text().lower()

    elif expected == 'invalid_credentials':

        assert 'do not match' in context.login_page.get_error_text().lower()

    else:
```

```
raise ValueError(f'Unknown expected outcome: {expected}')
```

6. Output

Running behave features/login.feature now executes 5 scenarios in total — the original Scenario from Assignment 10, plus one auto-generated scenario per Examples row in the Scenario Outline (4 rows). Each row saves its own screenshot (picB11_success_result.png, picB11_invalid_credentials_result.png, picB11_locked_out_result.png) before its assertion runs.

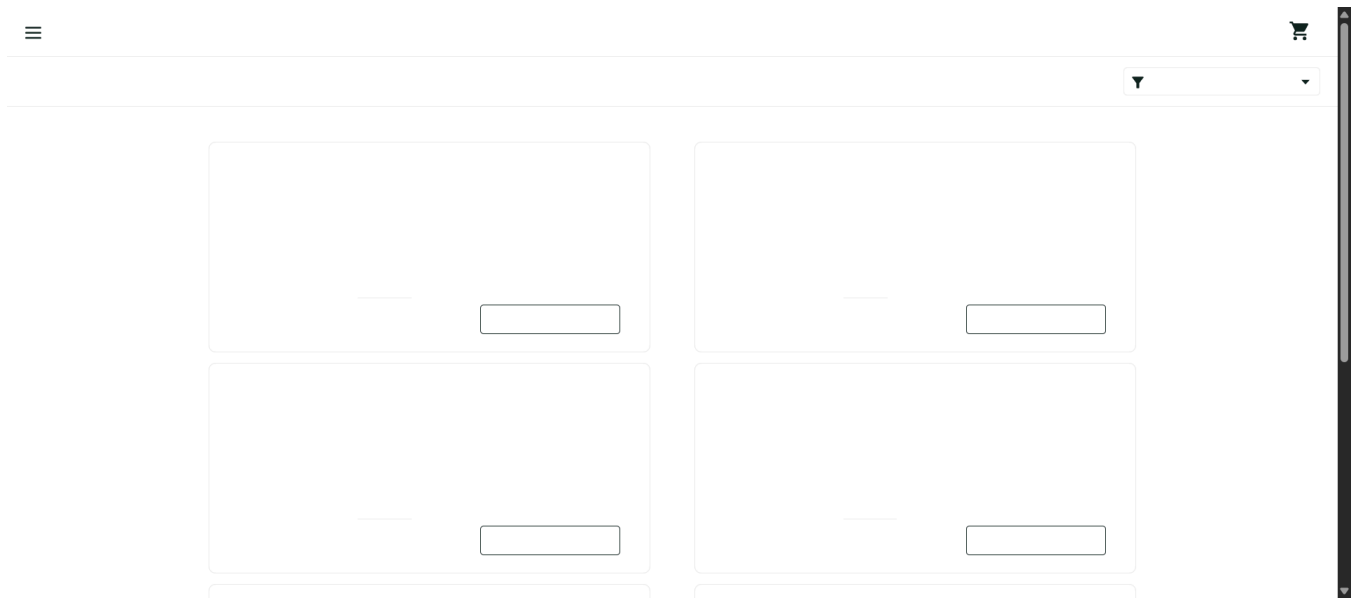


Fig B11.1 — picB11_success_result.png (auto-captured)

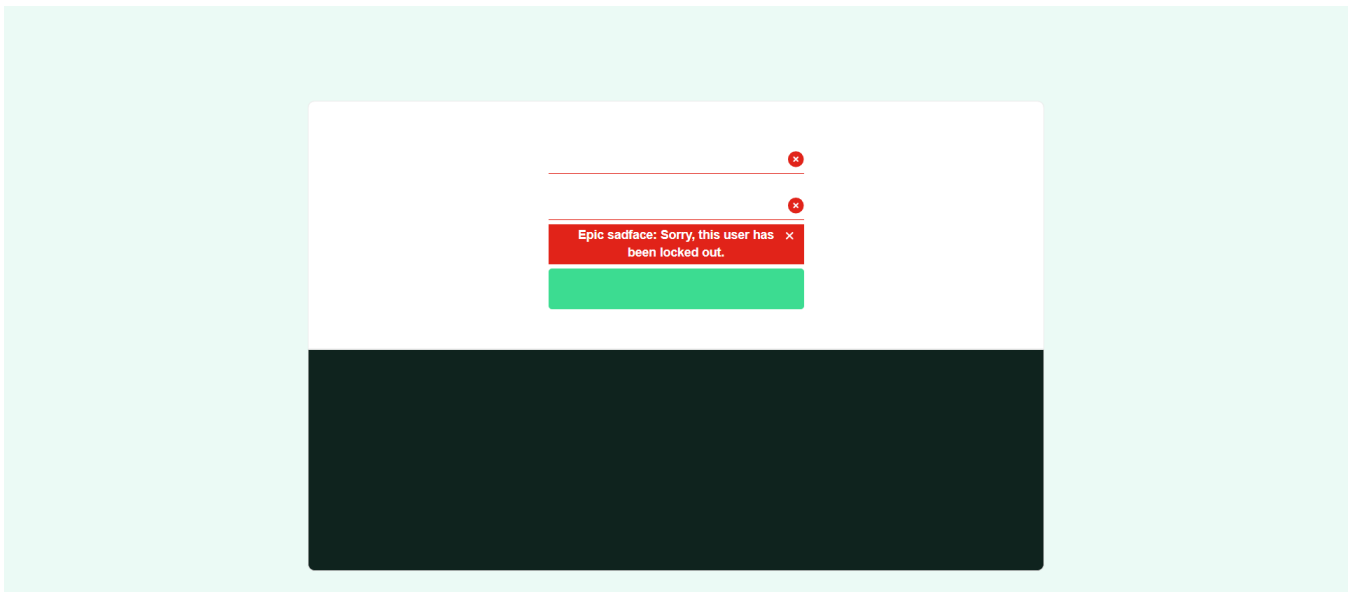


Fig B11.2 — picB11_locked_out_result.png (auto-captured)

```

As a registered SauceDemo user
I want to log in with valid credentials
So that I can reach the product inventory page
Then the user should land on the inventory page # features/steps/login_steps.py:14 0.135s
Then the user should land on the inventory page # features/steps/login_steps.py:14
Then the user should land on the inventory page # features/steps/login_steps.py:14 0.105s
Then the user should land on the inventory page # features/steps/login_steps.py:14
Then the login outcome should be "success" # features/steps/login_steps.py:23 0.122s
Then the login outcome should be "success" # features/steps/login_steps.py:23
Then the login outcome should be "invalid_credentials" # features/steps/login_steps.py:23 0.068s
Then the login outcome should be "invalid_credentials" # features/steps/login_steps.py:23
Then the login outcome should be "locked_out" # features/steps/login_steps.py:23 0.074s
Then the login outcome should be "locked_out" # features/steps/login_steps.py:23
Then the login outcome should be "invalid_credentials" # features/steps/login_steps.py:23 0.073s
Then the login outcome should be "invalid_credentials" # features/steps/login_steps.py:23

```

Fig B11.3 — Terminal output

7. Result

All four username/password combinations were exercised from a single Scenario Outline, with the correct success/error outcome verified and a screenshot captured automatically for every generated scenario.

8. Observation

- A Scenario Outline's Examples table is Gherkin's equivalent of Assignment 8's CSV file — Behave itself expands each row into an independent scenario, so one failing row is reported on its own without hiding the results of the other rows.
- Because the placeholders <username>, <password>, and <expected> are substituted directly into the step text, the same When step from Assignment 10 is reused unchanged for the data-driven scenarios — only one new Then step had to be added.
- Taking the screenshot before the assert (not in an except block) again guarantees evidence exists for every row, passing or failing, matching the approach used in Assignment 8.

9. Conclusion

The single hardcoded login scenario was converted into a genuinely data-driven Behave suite: adding a fifth credential combination now means adding one row to the Examples table, not writing a new scenario or step.

Assignment 12: Page Object Model Discipline & Automatic Failure-Screenshot Hook in Behave

Module	Module 3 – Python BDD RESTful Automations
Assignment Title	Page Object Model Discipline & Automatic Failure-Screenshot Hook in Behave
Sub-topic Ref.	19.4 – Scenario 3: Selenium Page Object Model in the Python Behave Framework

1. Problem Statement

Two gaps remain from Assignments 10–11: (1) there is no automated check that step definitions stay free of raw locators as the suite grows, and (2) unlike Assignment 9's PyTest suite, a failing Behave step does not automatically capture a screenshot unless a step happens to call `save_screenshot()` itself. The framework needs an `after_step` hook that captures evidence for ANY failure, plus a browser-viewable report, mirroring what `pytest-html` gave Assignment 9.

2. Objective

- Confirm `features/steps/login_steps.py` contains only Given/When/Then functions calling `LoginPage/InventoryPage` methods — no By locators, no raw `driver.find_element` calls.
- Implement an `after_step` hook in `environment.py` that automatically screenshots and attaches evidence whenever ANY step fails, independent of what that step's own code does.
- Generate a browser-viewable Allure HTML report with the failure screenshot embedded, the Behave equivalent of Assignment 9's `pytest-html` report.
- Include one deliberately-failing scenario, as Assignment 9 did, to prove the hook actually fires.

3. Tools, Software & Concepts Used

- `behave`
- `allure-behave` (`pip install allure-behave`) and the Allure commandline report viewer
- Behave hooks: `after_step`, `context.driver.get_screenshot_as_png()`
- Reused `LoginPage / InventoryPage` page objects from Assignment 7

4. Implementation Steps

11. Install the reporting package: `pip install allure-behave` (the Allure commandline tool is installed separately to view the report — see Anthropic-independent install docs for your OS).
12. In `features/environment.py`, add an `after_step(context, step)` hook that checks `step.status == 'failed'` and, if so, attaches a live screenshot via `allure.attach()`.
13. Re-read `features/steps/login_steps.py` end to end and confirm every function body calls only `context.login_page / context.inventory_page` — no locators.
14. Add one deliberately-failing scenario to `login.feature` that asserts the wrong outcome, purely to prove the `after_step` hook fires (remove once verified, exactly as Assignment 9 did).
15. Run with the Allure formatter: `behave -f allure_behave.formatter:AllureFormatter -o reports/allure-results features/`
16. View the report: `allure serve reports/allure-results` (opens the report in your default browser).

5. Source Code

features/environment.py (after_step hook added)

```
import time
import allure
from selenium import webdriver
from pages.login_page import LoginPage
from pages.inventory_page import InventoryPage

def before_scenario(context, scenario):
    context.driver = webdriver.Chrome()
    context.driver.maximize_window()
    context.login_page = LoginPage(context.driver)
    context.inventory_page = InventoryPage(context.driver)

def after_step(context, step):
    """Runs after EVERY step in EVERY scenario - automatically attaches
    a screenshot the moment a step fails, with no per-step code needed.
    This is the Behave equivalent of Assignment 9's
    pytest_runtest_makereport hook."""
    if step.status == 'failed':
        allure.attach(
            context.driver.get_screenshot_as_png(),
            name=f'failure_{step.name}',
            attachment_type=allure.attachment_type.PNG,
        )

def after_scenario(context, scenario):
    time.sleep(3) # MANUAL SCREENSHOT WINDOW (optional) - browser stays open
    context.driver.quit()
```

features/login.feature (deliberately-failing scenario appended)

```
Scenario: Deliberately wrong assertion to prove the failure-screenshot hook
# Intentionally asserts the WRONG outcome so the suite has a genuine
# FAILED step with an auto-attached screenshot to inspect in the Allure
# report (remove this scenario once the report has been verified).
```

```
Given the user is on the SauceDemo login page
When the user logs in with username "standard_user" and password "secret_sauce"
Then the login outcome should be "locked_out"
```

features/steps/login_steps.py — unchanged from Assignment 11, shown here to confirm no locators

```
from behave import given, when, then

@given('the user is on the SauceDemo login page')
def step_open_login_page(context):
    context.login_page.open() # no locator - delegates to LoginPage

@when('the user logs in with username "{username}" and password "{password}"')
def step_perform_login(context, username, password):
    context.login_page.login(username, password) # no locator

@then('the user should land on the inventory page')
def step_verify_inventory_loaded(context):
    loaded = context.inventory_page.is_loaded() # no locator
    context.driver.save_screenshot('picB10_inventory_loaded.png')
    assert loaded, 'Inventory page did not load after a valid login'

@then('the login outcome should be "{expected}"')
def step_verify_login_outcome(context, expected):
    context.driver.save_screenshot(f'picB11_{expected}_result.png')
    if expected == 'success':
        assert context.inventory_page.is_loaded(), \
            'Expected successful login but inventory page did not load'
    elif expected == 'locked_out':
        assert 'locked out' in context.login_page.get_error_text().lower()
    elif expected == 'invalid_credentials':
        assert 'do not match' in context.login_page.get_error_text().lower()
    else:
        raise ValueError(f'Unknown expected outcome: {expected}')
```

6. Output

Running `behave -f allure_behave.formatter:AllureFormatter -o reports/allure-results features/` executes all 6 scenarios (1 from Assignment 10, 4 from Assignment 11's Examples table, 1 deliberate failure). The deliberate-failure scenario's Then step fails, which fires the `after_step` hook and attaches a live screenshot to that scenario's entry in the Allure report — with zero screenshot code inside the failing step itself.

```
(.venv) PS C:\Users\swarn\Downloads\my-course-repo\PythonProject> behave -f allure_behave.formatter:AllureFormatter -o reports/allure-results features/
USING RUNNER: behave.runner:Runner

Failing scenarios:
  features/login.feature:28  Deliberately wrong assertion to prove the failure-screenshot hook

0 features passed, 1 failed, 0 skipped
6 scenarios passed, 1 failed, 0 skipped
20 steps passed, 1 failed, 0 skipped
Took 0min 11.968s
```

Fig B12.1 — Terminal output

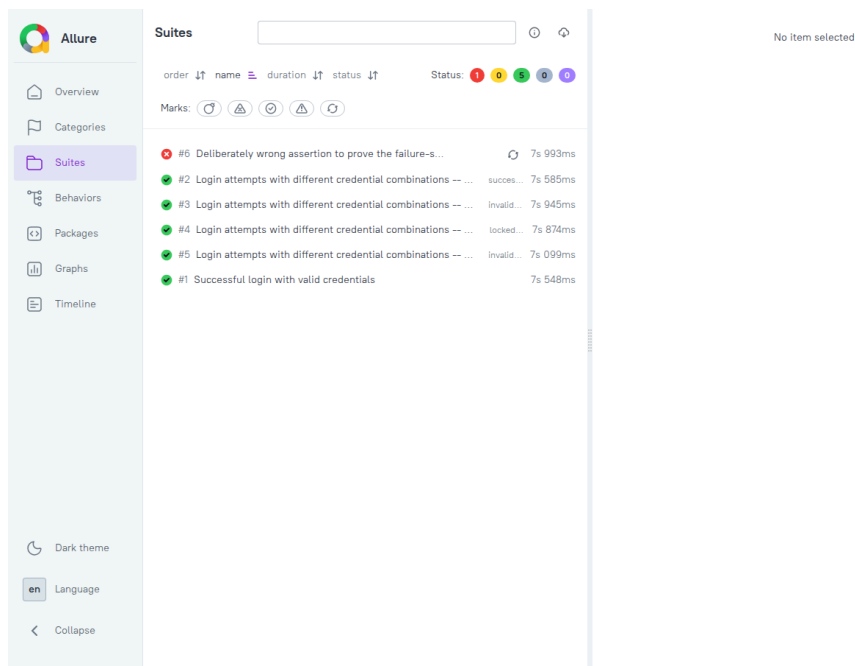


Fig B12.2 — Allure report, scenario list

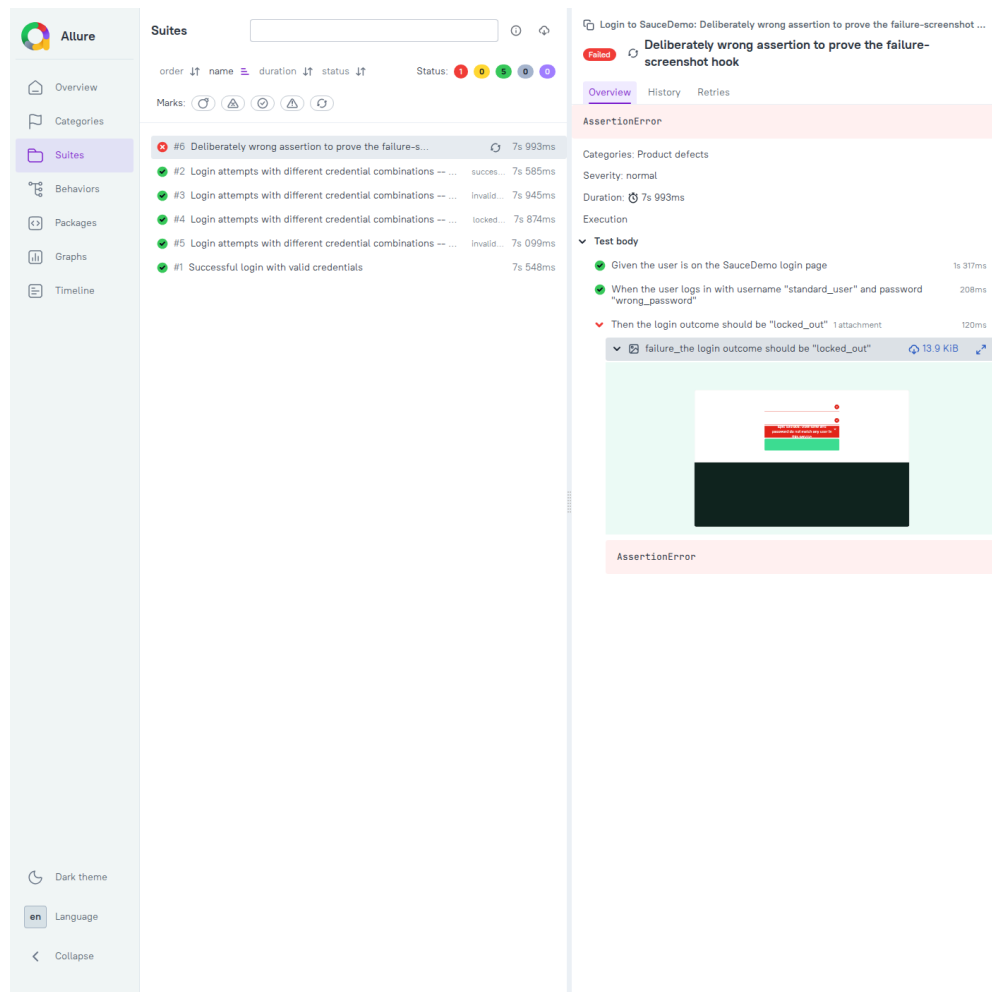


Fig B12.3 — Allure report, failed scenario expanded

7. Result

The Behave suite now automatically captures and attaches a screenshot for any step that fails, with no manual step required, and a browser-viewable Allure report confirms the login step definitions still contain zero raw locators after two assignments of growth.

8. Observation

- `after_step` runs after every step in every scenario in the whole project, so the failure-screenshot logic never has to be duplicated inside individual step definitions — new steps get the same safety net for free.
- Checking `step.status == 'failed'` specifically (rather than `scenario.status`) is what lets the hook point at the exact step that broke, instead of just flagging the whole scenario as failed.
- The deliberately-failing scenario exists only to prove the hook fires at all — the same reasoning Assignment 9 used for its intentional PyTest failure — and should be deleted once the Allure report has been checked.

9. Conclusion

Module 3 closes the same way Module 2 did: a login flow that started as a flat script is now running under a BDD framework with reusable Page Objects, data-driven Examples, and an automatic failure-reporting hook —

completing the progression from raw script → POM → data-driven testing → framework-level reporting, this time in Behave instead of PyTest.